

UNIVERSIDAD TECNOLOGICA NACIONAL

Tecnicatura Universitaria en Programacion

Programacion 2

Trabajo Practico Integrador

Food Store - Sistema de Gestion de Pedidos de Comida

Integrantes:

Leonel Ponce

Bruno Fiouchetta

Comision: 3

Profesores: Ramiro Hualpa - Cintia Rigoni

Año 2026 - Segundo semestre

Fecha de entrega: 22/06/2026

1. Introducción

El presente informe documenta el desarrollo del Trabajo Práctico Integrador de la materia Programación 2: Food Store, un sistema de consola para la gestión de pedidos de un negocio de comidas. El sistema permite administrar categorías, productos, usuarios y pedidos mediante operaciones de alta, baja, modificación y consulta (CRUD), con persistencia en una base de datos relacional MySQL a través de JDBC.

El desarrollo se realizó en equipo de dos integrantes, aplicando los conceptos de Programación Orientada a Objetos, manejo de excepciones, genéricos y el patrón DAO vistos a lo largo de la cursada. El control de versiones del código se llevó con Git y GitHub, repartiendo las tareas entre los integrantes.

2. Marco teórico

Para el desarrollo del proyecto se utilizaron las siguientes tecnologías y conceptos:

- **Java 21:** lenguaje principal del proyecto, orientado a objetos.
- **Programación Orientada a Objetos:** encapsulamiento mediante atributos privados con getters y setters, herencia (todas las entidades heredan de una clase abstracta Base), polimorfismo, clases abstractas e interfaces.
- **Colecciones:** uso de ArrayList para manejar listas de objetos, por ejemplo, los detalles de un pedido.
- **Manejo de excepciones:** se crearon excepciones propias (EmailDuplicadoException, EntidadNoEncontradaException y StockInvalidoException) para controlar los errores de negocio.
- **Bases de datos relacionales:** base de datos relacional con tablas vinculadas mediante claves foráneas (se utilizó MySQL/MariaDB a través de XAMPP).
- **JDBC:** API de Java para conectarse a la base. Se usaron Connection, PreparedStatement (consultas parametrizadas para evitar inyección SQL), ResultSet y try-with-resources para cerrar los recursos automáticamente.
- **Patrón DAO:** patrón de diseño que separa el acceso a datos del resto de la aplicación. Cada entidad tiene su DAO, que es el único lugar donde se escribe SQL.
- **Genéricos:** interfaces genéricas (IBaseDAO<T> e IGenericService<T>) para reutilizar las operaciones CRUD en todas las entidades sin repetir código.
- **Transacciones:** al crear un pedido con sus detalles se usa commit y rollback para mantener la consistencia de los datos.

3. Decisiones técnicas y arquitectura

El proyecto se organizó en capas para separar responsabilidades, siguiendo la estructura recomendada por la cátedra. La división en paquetes quedó de la siguiente manera:

- **entities:** clases del modelo (Base, Categoría, Producto, Usuario, Pedido, DetallePedido) y la interfaz Calculable.
- **enums:** enumerados Rol, Estado y FormaPago.

- **exception:** las excepciones propias del negocio.
- **config:** la clase ConexionDB, que centraliza la conexión a la base de datos.
- **dao:** las interfaces y las implementaciones del acceso a datos (los DAO).
- **service:** la lógica de negocio y las validaciones (los Service).
- **Menu / Main:** la clase Menu y el Main, que manejan la interacción con el usuario por consola.

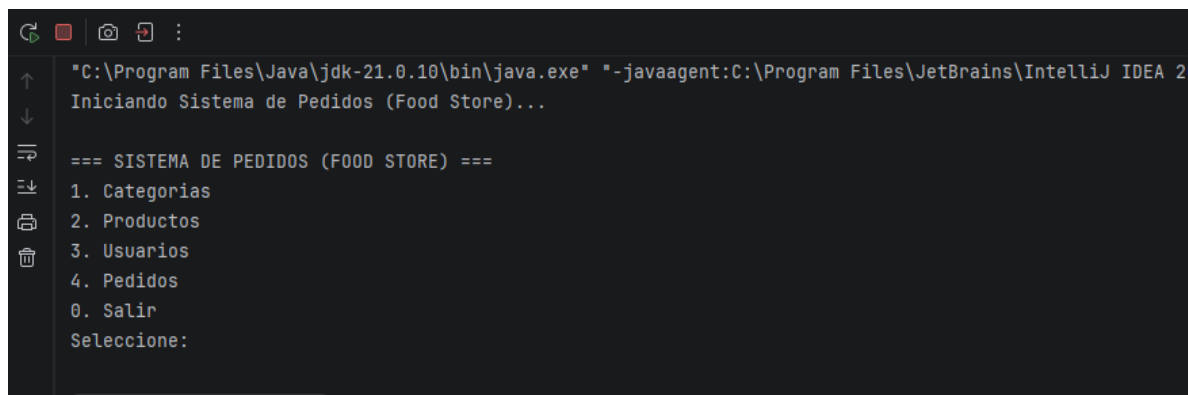
Cada capa tiene una responsabilidad clara: las entidades modelan el dominio, los DAO se encargan de la persistencia, los services validan las reglas de negocio (por ejemplo, que el mail de un usuario no se repita o que un producto no tenga precio negativo) y el menú maneja la entrada del usuario y muestra los resultados.

Decisiones de diseño más importantes:

- **Herencia con clase Base:** todas las entidades heredan de la clase abstracta Base, que aporta id, eliminado y createdAt, evitando repetir esos atributos en cada clase.
- **Baja lógica (soft delete):** en lugar de borrar físicamente un registro, se marca el campo eliminado = true. Las consultas filtran por eliminado = 0, de modo que se mantiene el historial.
- **Genéricos para el CRUD:** las interfaces IBaseDAO<T> e IGenericService<T> definen las operaciones comunes (guardar, obtener, listar, actualizar y eliminar) una sola vez para todas las entidades.
- **ConexionDB centralizada:** una única clase concentra los datos de conexión (URL, usuario y contraseña) para no repetirlos en cada DAO.
- **Transacción en el pedido:** al crear un pedido se validan el stock y los datos, se descuenta el stock y se guardan el pedido y sus detalles. Si algo falla, se realiza un rollback para no dejar la base en un estado inconsistente.

Capturas de pantalla del sistema funcionando:

Figura 1: Interfaz interactiva del Menú Principal por consola.

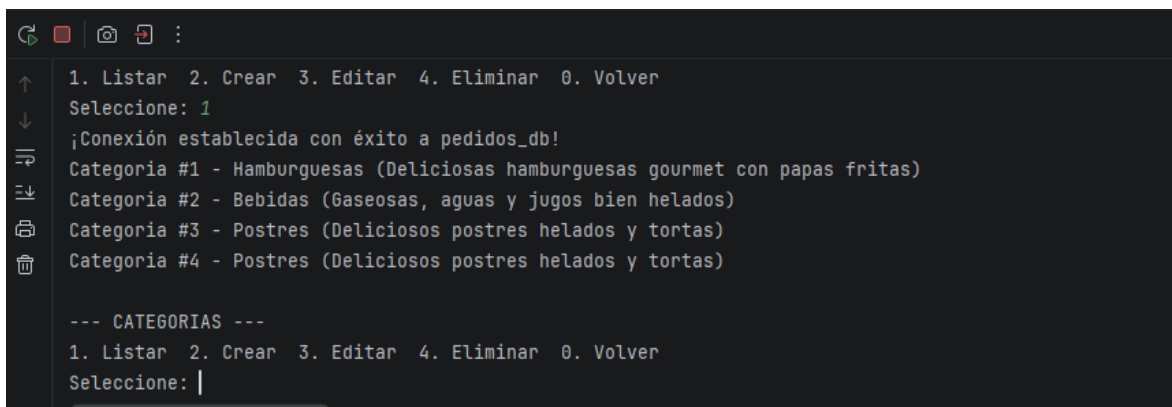


```

"C:\Program Files\Java\jdk-21.0.10\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2
Iniciando Sistema de Pedidos (Food Store)...

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione:
  
```

Figura 2: Operación de lectura (Listar) de categorías consumiendo datos de MySQL.



```
1. Listar  2. Crear  3. Editar  4. Eliminar  0. Volver
Seleccione: 1
¡Conexión establecida con éxito a pedidos_db!
Categoria #1 - Hamburguesas (Deliciosas hamburguesas gourmet con papas fritas)
Categoria #2 - Bebidas (Gaseosas, aguas y jugos bien helados)
Categoria #3 - Postres (Deliciosos postres helados y tortas)
Categoria #4 - Postres (Deliciosos postres helados y tortas)

--- CATEGORIAS ---
1. Listar  2. Crear  3. Editar  4. Eliminar  0. Volver
Seleccione: |
```

Figura 3: Creación interactiva de un pedido con múltiples detalles y persistencia transaccional (Commit) en la base de datos.

```

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 4

--- PEDIDOS ---
1. Listar 2. Crear 3. Editar estado/pago 4. Eliminar 0. Volver
Seleccione: 2
¡Conexión establecida con éxito a pedidos_db!
Usuario #1 - Leonel Ponce (leo.admin@foodstore.com) - ADMIN
Usuario #2 - Bruno Fioucheta (bruno.cliente@gmail.com) - USUARIO
Id del usuario: 1
¡Conexión establecida con éxito a pedidos_db!
Forma de pago: 1. TARJETA 2. TRANSFERENCIA 3. EFECTIVO
Seleccione: 2
¡Conexión establecida con éxito a pedidos_db!
Producto [ID=1, Nombre=Hamburguesa Simple, Precio=$4500.0, Stock=20, Disponible=true, Categoria=Hamburg
Producto [ID=2, Nombre=Hamburguesa Doble Bacon, Precio=$5800.0, Stock=20, Disponible=true, Categoria=Ha
Producto [ID=3, Nombre=Coca-Cola Original 500mL, Precio=$1500.0, Stock=50, Disponible=true, Categoria=B
Producto [ID=4, Nombre=Agua Mineral 500mL, Precio=$1200.0, Stock=40, Disponible=true, Categoria=Bebidas
Producto [ID=5, Nombre=Hamburguesa Triple Cheddar, Precio=$6500.0, Stock=15, Disponible=true, Categoria
Producto [ID=6, Nombre=Hamburguesa Triple Cheddar, Precio=$6500.0, Stock=15, Disponible=true, Categoria
Producto [ID=7, Nombre=Hamburguesa Triple Cheddar, Precio=$6500.0, Stock=15, Disponible=true, Categoria
Producto [ID=8, Nombre=Hamburguesa Triple Cheddar, Precio=$6500.0, Stock=15, Disponible=true, Categoria
Id del producto: 8
Cantidad: 2
Agregado. Total parcial: $13000.0
Agregar otro producto? (s/n): n
Producto con ID 8 actualizado con éxito.
¡Pedido #3 y todos sus detalles guardados con éxito mediante transacciones!
Pedido creado ok. Total: $13000.0

--- PEDIDOS ---
1. Listar 2. Crear 3. Editar estado/pago 4. Eliminar 0. Volver
Seleccione: |

```

4. Dificultades y soluciones

Durante el desarrollo se presentaron varios obstáculos técnicos. Los principales y cómo los resolvimos fueron:

- **Confusión con el alcance del trabajo:** al principio teníamos dudas de si el sistema debía usar base de datos o trabajar en memoria con colecciones, porque circulaban versiones distintas de la consigna. Lo resolvimos confirmando con la cátedra que la versión final usaba JDBC y MySQL.
- **Diferencia de nombres entre la base y Java:** en la base la columna se llama `created_at` y en Java el atributo es `createdAt`, por lo que al principio no mapeaba. Lo solucionamos haciendo la conversión en el método que arma el objeto a partir del `ResultSet` en cada DAO.

- **Versiones distintas del driver JDBC:** cada integrante había descargado una versión distinta del conector de MySQL (8.4.0 y 9.x). Para evitar problemas, unificamos todo a la versión 8.4.0 (LTS) y la subimos al repositorio.
- **Interfaces genéricas duplicadas:** trabajando en paralelo, cada uno creó una interfaz genérica para los services con métodos casi iguales pero distinto nombre. Lo resolvimos unificando todo en una sola interfaz (IGenericService).
- **Configuración del entorno (XAMPP):** tuvimos algunos enredos con XAMPP, como confundir el servidor MySQL con el cliente MySQL Workbench, y que phpMyAdmin no abría porque Apache estaba en un puerto distinto al estándar. Lo resolvimos accediendo por la dirección correcta y asegurándonos de iniciar el servidor MySQL desde el panel de XAMPP.

5. Bibliografía / Webgrafía

- Documentación oficial de Java (Oracle): <https://docs.oracle.com/en/java/>
- Tutorial de JDBC (Oracle): <https://docs.oracle.com/javase/tutorial/jdbc/>
- Documentación de MySQL: <https://dev.mysql.com/doc/>
- Apuntes y material de la cátedra de Programación 2 (UTN).
- Se utilizó asistencia de inteligencia artificial (Claude) como herramienta de apoyo para la resolución de dudas y la revisión del código.

6. Links del proyecto

- **Repositorio en GitHub:** <https://github.com/leo-pnc/food-store-tpi>
- **Video explicativo:** no corresponde (la cátedra confirmó que no era necesario para esta entrega).